

Entrega de Backend & API com Supabase

Playbook + checklist para projetos de cliente (gigs)

Cada fase entrega **software funcionando** E um **documento profissional** — a documentação está embutida no fluxo, não no fim.

Stack: Supabase (Postgres + Auth + RLS + Edge Functions). Sandro Abreu — Engenharia de Dados & Automação

Por que seguir um método

Cliente não paga caro por código — paga por **confiança**: que o dado dele está seguro, que a entrega funciona e que ele não fica dependente de você. Um processo repetível com documentação em cada etapa é o que transforma um freelancer em uma entrega de consultoria — e é o que justifica preço premium e torna o handover trivial.

Regra de ouro: **nunca documento no fim**. Cada fase produz seu documento enquanto a decisão está fresca. No fim, a documentação já existe.

Visão geral das fases

Fase	Objetivo	Documento produzido
0. Discovery	Entender o problema e montar o terreno	Requirements doc
1. Modelagem	Desenhar os dados (docs-first)	ERD, dicionário, convenções, ADRs
2. Schema	Criar as tabelas (SQL versionado)	Migrations + README de setup
3. Segurança	Auth + RLS (isolamento)	Estratégia de RLS + teste
4. API	Expor e a lógica de negócio	Mapa 'onde mora cada regra'
5. Integrações	IA, pagamentos, webhooks	Doc das Edge Functions
6. Entrega	Handover pro cliente	README de handover

FASE 0 — Discovery & Setup

Objetivo: entender o problema do cliente e preparar o terreno antes de qualquer código.

- [] Reunião de kickoff: quem usa? é multi-tenant (vários clientes isolados)? quais entidades e regras de negócio? algum dado sensível/regulado?
- [] Definir o escopo do MVP — cortar o que não é essencial pra v1.
- [] Criar a org/projeto no Supabase — **idealmente na conta do cliente** (billing e propriedade dele desde o dia 1).
- [] Criar o repositório Git + estrutura de pastas (docs/, supabase/migrations/).

Documento desta fase: Requirements doc — requisitos funcionais e não-funcionais, restrições do domínio. É o que o schema precisa suportar.

FASE 1 — Modelagem de dados (docs-first)

Objetivo: desenhar os dados antes do SQL. O schema é consequência da modelagem, não o contrário.

- [] Definir **todas as entidades** (nível conceitual) antes de pensar em propriedades.
- [] Definir propriedades, tipos, chaves e relações de cada entidade.
- [] Fechar as convenções de nomenclatura (singular/plural, snake_case, FKs, enums).
- [] Registrar cada decisão não-óbvia em um **ADR** (por que X e não Y).
- [] Desenhar no **dbdiagram.io** (DBML) pra validar visualmente e com o cliente.

Documento desta fase: Modelo de dados (ERD), dicionário de dados (tabela por tabela, coluna por coluna), **convenções de nomenclatura** e os **ADRs** das decisões.

FASE 2 — Schema (SQL versionado)

Objetivo: transformar o desenho em tabelas reais, de forma reproduzível.

- [] Traduzir o DBML em SQL, **camada por camada** (referência -> cadastro -> transacional), respeitando as dependências de FK.
- [] Rodar no SQL Editor do Supabase (ou via supabase CLI).
- [] Versionar como **migrations no Git** (supabase/migrations/NNNN_*.sql) — é a sua 'receita' pra levantar o mesmo banco em qualquer projeto/cliente.
- [] Semear os dados de referência (ex: catálogos, tabelas fixas).
- [] Tipos: uuid PK com default gen_random_uuid(); timestampz default now(); enums como text + check.

Documento desta fase: Migrations versionadas (o próprio SQL é documentação executável) + **README de setup** (como subir o banco do zero).

FASE 3 — Segurança (Auth + RLS)

Objetivo: garantir que cada um só acessa o que é seu. Em multi-tenant, isto é obrigatório.

- [] Configurar o **Supabase Auth** (sign-up / login).
- [] **Ligar RLS em TODAS as tabelas** (secure by default — o Supabase alerta se uma tabela do public ficar sem RLS).
- [] Tabelas de **referência** (globais): política de leitura aberta pra autenticados; escrita só por service_role.
- [] Tabelas do **tenant**: políticas de isolamento (helper que resolve 'minhas contas' via auth.uid()).
- [] **Testar o isolamento**: 2 contas fake; confirmar que A não vê dado de B.

Lembrete: RLS = negar por padrão; cada política é uma **chave** que abre uma porta específica. service_role (admin) ignora o RLS — use só no servidor, nunca no client.

Documento desta fase: Estratégia de RLS (quais tabelas são referência vs tenant, o padrão de política) + **plano de teste de isolamento**.

FASE 4 — A API e a lógica de negócio

Objetivo: expor os dados e colocar as regras nos lugares certos. No Supabase você quase não escreve API de CRUD.

- [] A **API REST é auto-gerada** (PostgREST) a partir do schema — não escreva controllers de CRUD à mão.
- [] Regras de **integridade** (ex: campo obrigatório condicional): constraints / CHECK no banco.
- [] Regras **reativas** (ex: imutabilidade, cálculo de campo): triggers + funções.
- [] Operações **transacionais complexas**: funções Postgres expostas como RPC.
- [] Testar os endpoints (leitura/escrita) com um usuário autenticado.

Documento desta fase: Mapa 'onde mora cada regra de negócio' — o que está no banco (constraint/trigger/função) vs o que está em Edge Function. Auditável.

FASE 5 — Lógica de servidor & integrações

Objetivo: o que precisa de servidor de verdade (segredos, serviços externos).

- [] **Edge Functions** (TypeScript) para: IA, Stripe, webhooks, n8n, e qualquer coisa que use chave secreta.
- [] Nunca expor a **service_role** no client — ela vive só no servidor.
- [] No navegador use a chave **anon** — ela é segura porque o RLS protege.
- [] Validar cada integração ponta a ponta.

Documento desta fase: Doc das Edge Functions / integrações — o que cada função faz, quais segredos usa, e o fluxo.

FASE 6 — Entrega & handover

Objetivo: passar pro cliente de forma limpa, sem deixá-lo dependente de você.

- [] Revisar a documentação final (já produzida nas fases anteriores).
- [] Garantir que as migrations sobem do **zero num projeto limpo** (reprodutibilidade real).
- [] Transferir o projeto / confirmar que o **billing e a propriedade são do cliente**.
- [] Entregar o **README de handover**: como rodar, deployar e manter.

Documento desta fase: README de handover — visão geral, como subir o ambiente, deploy, e onde está cada coisa.

Checklist resumido (marque por projeto)

- ☐ Kickoff + requirements doc revisado com o cliente
- ☐ Projeto Supabase na conta do cliente + repo Git criado
- ☐ Todas as entidades definidas (conceitual) antes das propriedades
- ☐ Dicionário de dados + convenções + ADRs escritos
- ☐ DBML validado no dbdiagram.io
- ☐ Schema em SQL, camada por camada, versionado como migrations
- ☐ Dados de referência semeados
- ☐ Auth configurado
- ☐ RLS ligado em TODAS as tabelas
- ☐ Referência: leitura aberta; tenant: isolamento
- ☐ Isolamento testado com 2 contas
- ☐ Regras de negócio no lugar certo (banco vs Edge Function) + documentadas
- ☐ Edge Functions / integrações (IA, Stripe, webhooks) testadas
- ☐ Migrations sobem do zero num projeto limpo
- ☐ Projeto transferido / billing do cliente + README de handover

Referência rápida

Onde mora cada regra (mapa mental Spring -> Supabase)

No Spring você faria...	No Supabase fica...
@Repository (CRUD)	API REST automática (PostgREST)
Spring Security / filtros	RLS policies (por linha)
@Service: validação	CHECK / constraints + triggers
@Service: operação transacional	função Postgres (RPC)
chamar API externa / segredos	Edge Function (server-side)

Padrão de política RLS

Referência (leitura aberta):

```
create policy "ref_read" on standards
  for select to authenticated using (true);
```

Raiz do tenant (comparação direta):

```
create policy "own" on clients for select
  using (account_id in (select current_account_ids()));
```

Filha (join até a raiz):

```
create policy "own" on locations for select using (
  exists (select 1 from clients c
    where c.id = locations.client_id
    and c.account_id in (select current_account_ids())));
```

Princípios que não mudam

- Documento em cada fase, nunca só no fim.
- RLS ligado em tudo; abrir portas só com políticas.
- service_role só no servidor; anon no navegador (RLS protege).
- Migrations versionadas = handover trivial e zero lock-in (é Postgres puro).
- Cliente dono do projeto/billing desde o começo.

Playbook de entrega — Supabase API. Use como checklist a cada projeto.